Test-Driven Development

Diese Übung basiert auf dem Repository m426-2026-ELA/math bzw. auf deinem Fork davon.

In dieser Übung soll die Funktionalität erweitert werden.

Halte dich dabei an das TDD-Vorgehen (PDF) und verwende keine KI-Werkzeuge zum Generieren von Code! Hier geht es um das Einüben der Arbeitstechnik.

Teil 1: Brüche kürzen

Brüche können gekürzt werden, indem man Zähler und Nenner durch den gleichen Faktor teilt:

$$\frac{18}{27} = \frac{2 \cdot 9}{3 \cdot 9} = \frac{2}{3}$$

Den kleinstmöglichen Zähler und Nenner erhält man durch deren Division mit ihrem grössten gemeinsamen Teiler (ggT). Zum Kürzen eines Bruches muss also zunächst der ggT von Zähler und Nenner ermittelt werden.

Erstelle für die folgenden beiden Aufgaben zwei neue Dateien `gcd.ts` und `gcd_test.ts`.

Aufgabe 1: Grösster Gemeinsamer Teiler (Brute Force)

Schreibe eine Funktion `gcdBruteForce(a: number, b: number): number`. Diese soll den ggT von `a` und `b` durch systematisches Auspropieren ermitteln.

Prüfe dabei für den Zahlenbereich von $min(\frac{a}{2}, \frac{b}{2}) \geq i \geq 1$, ob i die Zahlen a und b restlos teilt `a % i == 0` bzw. `b % i == 0`). Das erste gefundene Ergebnis ist der ggT.

Hinweis: Schreibe zuerst triviale Tests, z.B. für `ggT(1, 1)`, bevor du die eigentliche Logik implementierst.

Aufgabe 2: Bruch kürzen

Schreibe eine Methode `Fraction.cancel()`, welche den Bruch kürzt. Sie soll den bestehenden Bruch bzw. die Eigenschaften `numerator` und `denominator` anpassen.

Hinweis: Schreibe wiederum zuerst triviale Tests, z.B. für den Bruch $\frac{1}{1}$ mit `ggT(1,1)=1`. Verwende für die Implementierung die vorher geschriebene Funktion `gcdBruteForce`.

Aufgabe 3: Grösster Gemeinsamer Teiler (Euklid-Algorithmus)

Der Brute-Force-Algorithmus aus Aufgabe 1 ist weder effizient noch elegant. Ergänze `gcd.ts` um eine Funktion `gcdEuclid(a: number, b: number): number`, welche den Algorithmus von *Euklid* umsetzt:

- Gegeben sind die Zahlen **a** und **b**.
- Berechne **c** als Ergebnis der Subtraktion der kleineren von der grösseren der beiden Zahlen **a** und **b**: **c=max(a,b) −min(a,b)**
- Wiederhole den Algorithmus mit **a=min(a,b)** und **b=c**, solange **a≠b**.
- Sind die beiden Zahlen **a** und **b** gleich, ist der ggT gefunden.

Hinweis: Dieser Algorithmus lässt sich sehr elegant rekursiv implementieren.

Stelle sicher, dass die Funktion `gcdEuclid` die Testfälle für `gcdBruteForce` erfüllt. Anschliessend kannst du die Implementierung von `Fraction.cancel()` entsprechend umstellen. (Die Tests müssen weiterhin funktionieren!)

Variante: Table-Driven Tests

Du kannst die Testfälle in eine Datenstruktur im Testmodul `gcd_test.ts` auslagern, beispielsweise folgendermassen:

```
const gcdTests = [
  { a: 1, b: 1, gcd: 1 },
  { a: 1, b: 2, gcd: 1 },
  { a: 2, b: 2, gcd: 2 },
  { a: 3, b: 4, gcd: 1 },
  { a: 6, b: 9, gcd: 3 },
  { a: 81, b: 36, gcd: 9 },
];
```

Diese arbeitest du dann in der Testfunktion folgendermassen ab:

```
for (const { a, b, gcd } of gcdTests) {
  const actual = gcdEuclid(a, b);
  assertEquals(actual, gcd)
}
```

Dieses Muster bezeichnet man als Table-Driven Tests (tabellengetriebene Tests). Es ist eine Alternative zum Muster Arrange, Act, Assert bzw. Given, When, Then, das vor allem dann sinnvoll ist, wenn es sehr viele Testfälle mit der gleichen Struktur gibt. (Verschiedene Testframeworks unterstützen diese Art des Testens mit spezialisierten APIs.)

Der Vorteil von Table-Driven Test ist, dass ein zusätzlicher Testfall nur einen weiteren Eintrag in die Testtabelle benötigt. Der Nachteil ist, dass der Testcode etwas komplizierter wird.

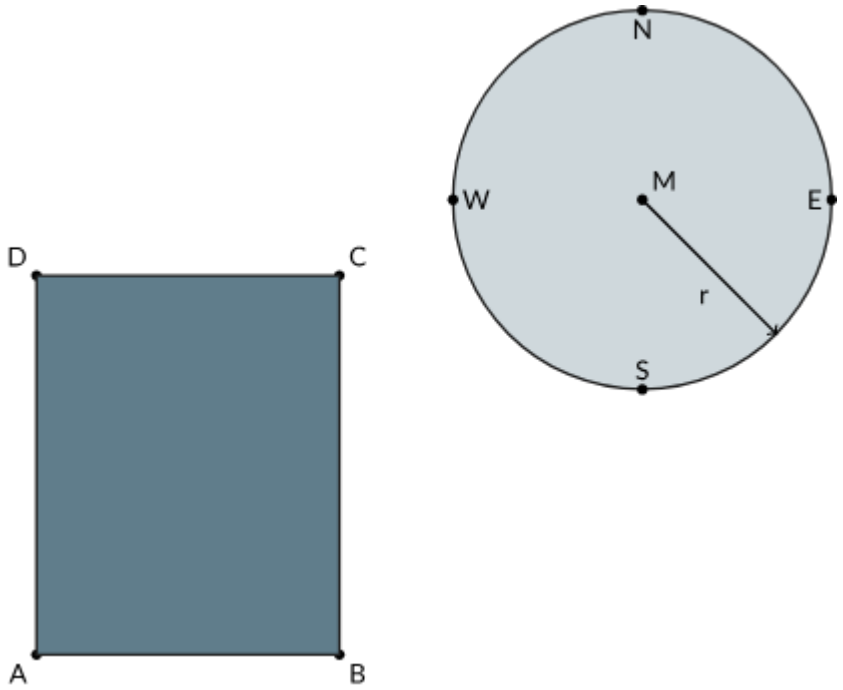
Aufgabe 4: Automatische Kürzung

Die Kürzung des Bruches kann bereits nach jeder Rechenoperation erfolgen oder auch im Konstruktor bzw. beim Parsen `Fraction.parse`) erfolgen.

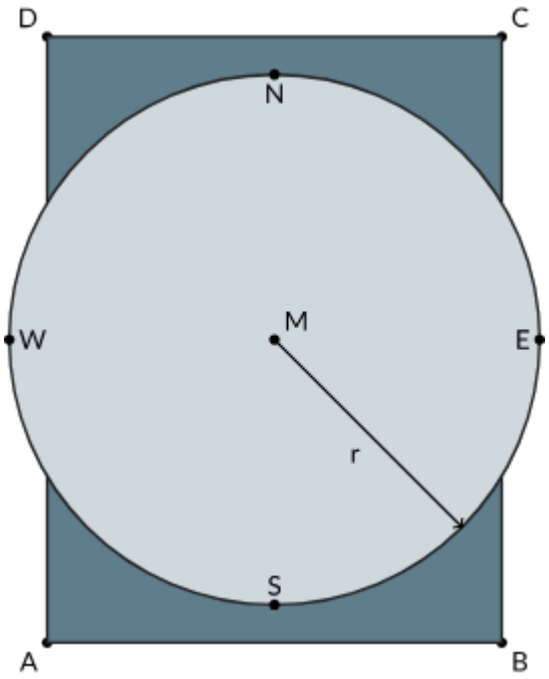
Implementiere die automatische Kürzung von Brüchen. Halte dich dabei ans TDD-Vorgehen, indem du zuerst scheiternde Testfälle schreibst, welche ein gekürztes Ergebnis erwarten. Passe anschliessend die Implementierung von `Fraction` an, damit die Testfälle (wieder) durchlaufen.

Teil 2: Umschliessende Objekte

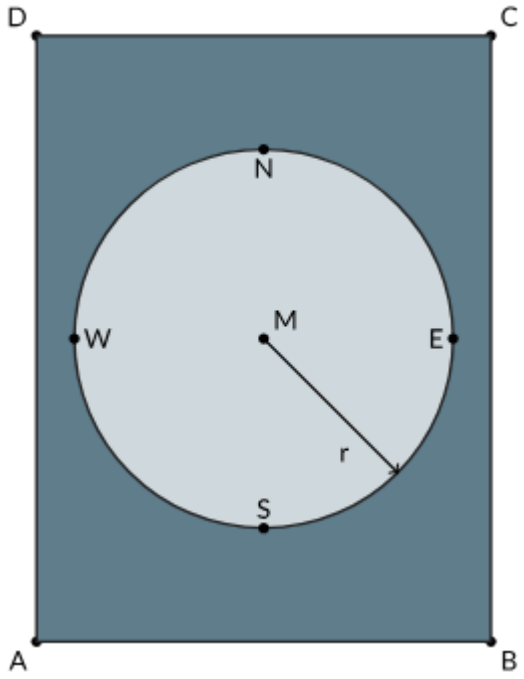
Die Schnittstelle `Shape` soll um eine Operation namens `encompasses` erweitert werden. Diese soll überprüfen, ob ein geometrisches Objekt ein anderes komplett enthält bzw. umschliesst.



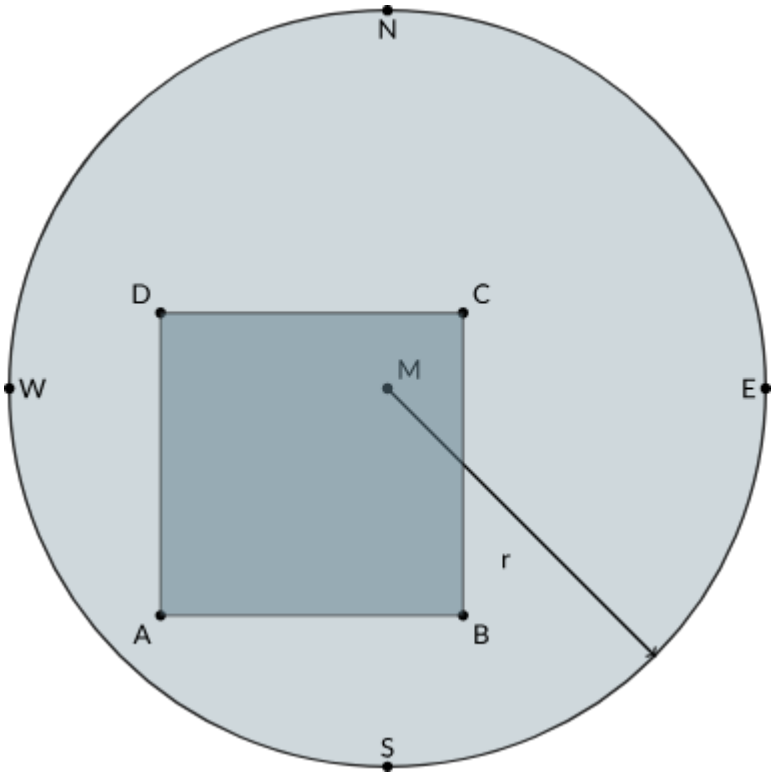
Die folgende Abbildung zeigt ein Rechteck und einen Kreis, die sich nicht einmal berühren und darum auch nicht umschliessen:



In der nächsten Abbildung liegt zwar der Kreismittelpunkt innerhalb des Rechtecks, der Radius des Kreises reicht aber über das Rechteck hinaus:



Ist der Radius kleiner, umfasst das Rechteck den Kreis komplett:



In der folgenden Abbildung ist die Situation umgedreht: Der Kreis enthält das Rechteck vollständig:

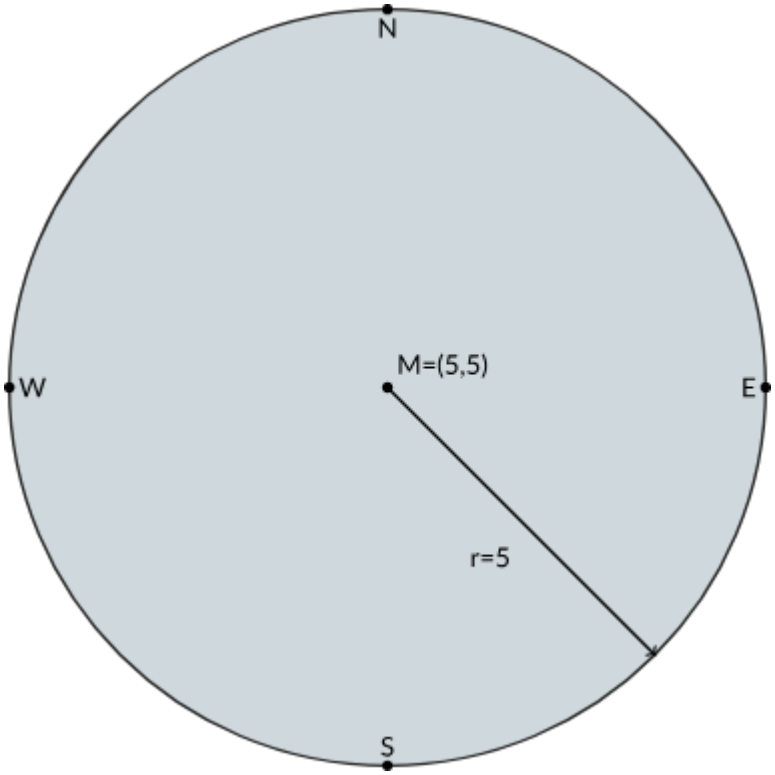
Erweitere in `geometry.ts` die Schnittstelle `Shape` um folgende Deklaration: `Shape.encompasses(other: Shape): boolean` . Diese Methode soll `true` zurückgeben, wenn das jeweilige Objekt das andere komplett umschliesst. Andernfalls soll `false` zurückgegeben werden.

Hinweis: Implementiere eine entsprechende Dummy-Methode für die Klassen `Rectangle` und `Circle` , damit es keine Kompilierfehler gibt.

Bearbeite anschliessend die folgenden Aufgaben zur Implementierung dieser Methode, wobei du dich ans TDD-Vorgehen hältst.

Aufgabe 5: Hilfsmethoden für Kreispunkte

Betrachte die folgende Abbildung eines Kreises mit dem Mittelpunkt *M* auf den Koordinaten (**5,5**) und dem Radius **r=5**:



Vom Kreis (`Circle`) ist zunächst nur dessen Mittelpunkt M (`center: Point2D`) und Radius r (`radius: number`) bekannt. Erweitere die Klasse `Circle` um die folgenden Methoden:

- `north(): Point2D`
- `east(): Point2D`
- `south(): Point2D`
- `west(): Point2D`

Diese sollen anhand des Mittelpunkts und Radius die entsprechenden Punkte ***N***, ***E***, ***S*** und **W** ermitteln und zurückgeben.

Hinweis: Die Punkte ***N*** und ***S*** haben die gleiche x-Koordinate wie ***M***, die Punkte ***E*** und ***W*** die gleiche y-Koordinate wie ***M***. Die y-Koordinaten von ***N*** und ***S*** sind entsprechen der y-Koordinate von ***M*** um den Kreisradius ***r*** erhöht oder verringert: $N_y = M_y + r$ bzw. $S_y = M_y - r$. Für die x-Koordinaten von ***W*** und ***E*** erfolgt die Rechnung analog dazu.

Aufgabe 6: Hilfsmethoden für Punkteinschluss

Erweitere die Klasse `Point2D` um folgende Methoden:

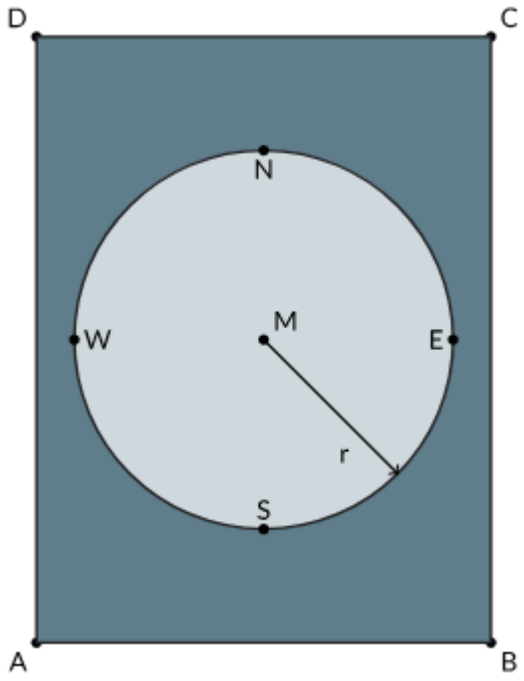
- `isBetweenX(p:Point2D, q: Point2D): boolean` : prüft, ob die Punkte p und q den Punkt auf der x-Achse einschlissen, sodass gilt p.x<x<q.x
- `isBetweenY(p: Point2D, q: Point2D): boolean` : prüft, ob die Punkte p und q den Punkt auf der y-Achse einschlissen, sodass gilt p.y<y<q.y

Hinweis: Als Testfälle kannst du die Methode auf dem Kreismittelpunkt ***M*** mit den Punkten ***N***, ***E***, ***S*** und ***W*** verwenden, wobei ***M*** auf der x-Achse zwischen ***W*** und ***E*** bzw. auf der y-Achse zwischen ***N*** und ***S*** liegt. Schreibe auch Negativtests dazu.

Aufgabe 7: Rechteck beinhaltet Kreis

Implementiere die Methode `Rectangle.encompasses(other: Shape): boolean` gemäss TDD-Vorgehen für Kreise (`other instanceof Circle`).

Betrachte hierzu noch einmal die folgende Abbildung:



Das Rechteck umschliesst den Kreis komplett, wenn sowohl dessen Mittelpunkt als auch die Extrempunkte ***N***, ***E***, ***S*** und ***W*** (für North, East, South, West) innerhalb des Rechtecks legen. Betrachte hierzu die x- und y-Achsen separat:

- die x-Koordinate vom Kreispunkt ***K*** muss zwischen den x-Koordinaten der Punkte ***A*** und ***B*** liegen: $A_x < K_x < B_x$
- die y-Koordinate vom Kreispunkt K muss zwischen den y-Koordinaten der Punkte ***A*** und ***D*** liegen: $A_y < K_y < D_y$

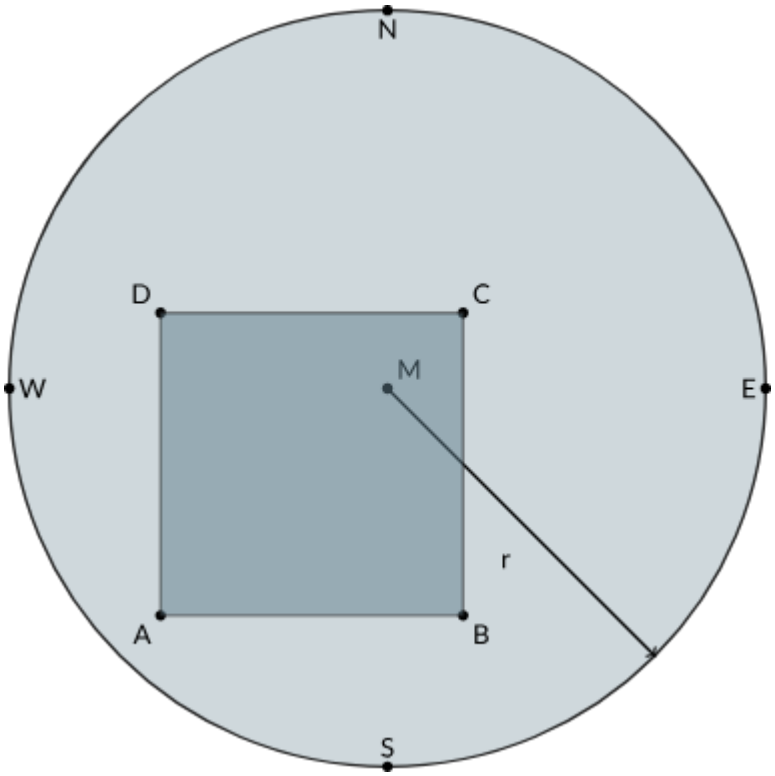
Als Kreispunkte ***K*** müssen die konkreten Punkte ***M***, ***N***, ***E***, ***S*** und ***W*** geprüft werden.

Hinweis: Verwende hierzu die Hilfsmethoden der vorherigen beiden Aufgaben.

Aufgabe 8: Kreis beinhaltet Rechteck

Implementiere die Methode `Circle.encompasses(other: Shape): boolean` gemäss TDD-Vorgehen für Rechtecke (`other instanceof Rectangle`).

Betrachte hierzu noch einmal die folgende Abbildung:



Der Kreis umschliesst das Rechteck komplett, wenn die Punkte ***A***, ***B***, ***C***, ***D*** alle innerhalb des Kreises liegen. Dies ist der Fall, wenn die Distanz zwischen ***M*** und dem jeweiligen Punkt kleiner ist als der Radius.

Hinweis: Die Distanz zwischen zwei Punkten kann über die Methode `Point2D.distanceTo(other: Point2D)` ermittelt werden.

Last updated 2026-03-26 19:59:31 +0100